

What the Axes Are Worth:

measurements on navigation and algorithmic tasks

Abstract

A companion to *Positional Mechanisms in Attention*, which classifies the space of positional operators and finds two live slots. This paper measures what two properties of those slots are worth — properties that are not applied as design constraints in the positional-encoding literature for language, and that no one has tested on navigation.

The **sign** of the phase increment costs nothing and decides what the accumulator measures: signed increments cancel, so the interval sum is net displacement and the code is a map; monotone ones cannot, so it is elapsed path and the code is a clock. On a map task, at $4\text{--}8\times$ the training length, a signed increment beats an index code by 0.12–0.20 at matched loss on 12/12 seeds and a monotone one beats it nowhere; at the training length the monotone arms still beat the index code, so the accuracy cost is an extrapolation effect. On a k -back recency task the cost of monotonicity vanishes, and the same unconstrained architecture learns a different accumulator for each task. At **input-side rank 2** of the content-to-phase map, training rarely *finds* the solution, although it exists: trained and tested at $T = 1024$ for 900 epochs from matched initial weights, our shared $r = 2$ solves 0/8 seeds, a per-head $r = 2\ 2/8$ and a shared $r = 4\ 8/8$, while a rank-2 solution exists (0.9955) and holds under training (measured for our shared $r = 2$). A separation batch identifies the responsible property as the per-head rank of the map: a block-diagonal $r = 4$ (rank 2 per head) solves 2/8 and a per-head $r = 4\ 8/8$ (Fisher and permutation $p = 0.0070$); cross-head sharing and W_{out} ’s per-entry scale are individually unmeasured.

Both are measured on grid navigation and algorithmic tasks rather than on text, because they are separable only where position must be reconstructed from the input rather than read off the index. Every evaluation is on an environment redrawn at test time, which makes them transfer measurements: at the training length a structural code reaches 0.97–1.00 where a parameter-matched index code reaches 0.81 (floor 0.506), and the gap grows with length.

We also report what did not work. An explicit content gate separates action from observation tokens by a factor of four and improves accuracy nowhere; a published external benchmark does not discriminate the mechanism, for a reason our own per-offset data predicted; and the growth exponent that unifies the sign and rank results is nearly collinear with a simpler observable and does not, on the evidence here, explain either.

Corrected 2026-09-25. Earlier versions quoted a torus position effect of +0.461 with the index code on the blank floor (a 16-epoch recipe; converged it is +0.243 with the index code at 0.805), explained the rank result by a badly conditioned $r = 2$ basis (within $r = 2$ the skew does not predict accuracy; the matched-length result above replaces it), and called recency a task “built to want a clock” on which the ordering “inverts” (a signed rewind solves it exactly, and nothing inverts).

Corrected 2026-09-27. The 2026-09-25 text said which property of the rank bottleneck is responsible was not separated; it is per-head rank (RANK_SEP_RESULTS.md). A search-aid batch at $r = 2$ (LOOP_RANK_RESULTS.md) is added to §3.2.

1 What is measured, and how

The axes referred to throughout — A1 factor, A2 locus, A3 driver, A4 algebra, A5 sign, A6 rank, A7 host — are those of the companion review, and the relational map placing every mechanism on them is Table 4 there. This paper assumes that classification and tests two of its cells.

Standard errors are over seeds; MDE is $2.8 \text{ sd} / \sqrt{n}$, and a contrast inside its MDE is reported as *unmeasured* rather than as a null. Contrasts are paired per seed, and arms compared against one another were trained in one batch. Where accuracy correlates with final training loss — it has been as strong as $r = -0.999$ — contrasts are given raw and loss-matched, and both are reported.

2 Factorisation, and what the separation buys

TEM’s thesis is a factorisation: structural knowledge g is environment-invariant, sensory input x is environment-specific, and memory stores the conjunction $p = g \otimes x$. Transfer follows because g survives a change of environment while x does not.

MapFormer is that factorisation carried into attention with a parallel scan. The accumulated phase $\theta_t = \omega \sum \Delta_u$ is g : identical in any environment with the same transition structure, because it depends on the action stream and not on what was seen. Content is x . And the conjunction is exactly the WM/EM distinction — for MapFormer-WM a kernel whose amplitudes and phases the pair’s content sets (measured phase spread 1.947 rad across pairs, against 0.000 for MapEM; note an untrained MapWM is MORE spread, at 2.722), and for MapEM *literally* the tensor product, since $(A_X \odot A_P)_{ts} = (q_t^x \otimes q_t^p) \cdot (k_s^x \otimes k_s^p)$. TEM’s $g \otimes x$ and MapEM’s Hadamard product are the same object; what MapFormer adds is that the interval operator composes by prefix sum, so the where can be built in $O(\log T)$ instead of recurrently.

But that separation is learned, not built — the difference between a factorisation the architecture guarantees and one it merely tends to find. MapFormer’s increment is computed from *every* token embedding — $\Delta_u = W_{\text{out}} W_{\text{in}} x_u$ with no gating by token type — so the model has to *discover* that $\Delta \approx 0$ on observations. It does: on a trained model actions move the phase about five times as much as observations do. And the recency result of §3.8 is the same phenomenon isolated — an unconstrained increment learns a content gate in 8/8 seeds, and a magnitude-matched intervention shows the gate is most of the mechanism rather than a component of it. So the separation is real and measurable; it is emergent rather than enforced.

And that is the design, not an oversight. MapFormer’s own text is explicit: “to learn a cognitive map, the model has to identify that action tokens update the internal position, while ensuring that observations leave the structure untouched. This is achieved through a two-stage projection process” — the low-rank $W_{\text{out}} W_{\text{in}}$ is the identification mechanism, with W_{in} mapping each token to “a low-dimensional action representation $\Delta_t \in \mathbb{R}^r$... capturing the actual motion to perform”. Hard-coding the split would remove exactly what the mechanism is for, and would confine it to settings with a labelled action stream — which excludes text, where there are no action tokens and RoPE is the $\Delta \equiv 1$ special case.

This has a consequence for §3.2 worth stating: **the bottleneck is the separator**. The rank r is not merely the dimension of a displacement code, it is the channel through which the what/where distinction has to pass, so the finding that training rarely finds the torus solution through a rank-2 bottleneck (§3.2) may be a finding about the *separation* and not only about the geometry. An enforced split is therefore an oracle *control* rather than a proposal: it upper-bounds what the learned separation can reach, and the gap measures what learning the split costs. If the two match, the design is vindicated with a number.

Why the measurements are transfer measurements. This is easy to miss, and it is the reason the evaluations throughout are set up as they are. **Every evaluation here is on an environment the model has never seen:** the observation map is redrawn from a held-out seed, so the entire what-to-where binding is new, and only structure can carry over. Against a measured always-predict-blank floor of 0.506:

Table 1: Transfer to a new environment with the same relational structure. In every row the sensory map is redrawn at evaluation, so nothing environment-specific survives; the gap is what the structural code carries. Torus: $T=128$ (the training length), converged recipe; structural = MapPoPE / MapFormer, index = RoPE / PoPE. The compositional row is at the old linear-decay recipe, which limits it (warmup and cosine lift the structural arm to 0.514), and its structural arm is hierarchical while the index arm is flat.

task	structural code	index code	floor
torus revisit, fresh map ($n=8$)	0.999 / 0.971	0.805 / 0.679	0.506
blind continuation, 64^2 ($n=5$)	0.730 $\pm$ 0.247	0.154 ± 0.018	0.063
blind continuation, 128^2 ($n=3$)	0.823 $\pm$ 0.043	0.192 ± 0.022	0.063
compositional motif transfer ($n=8$)	0.415 $\pm$ 0.096	0.216 ± 0.004	≈ 0.072

On the torus an index code is well above the floor at the training length once trained to convergence (0.805) but trails the structural code by 0.17–0.19, and falls to 0.374 at $8\times$ the training length where the structural code holds 0.78–0.92. (Earlier versions of this table gave 0.530 / 0.509, from a 16-epoch recipe under which the index code never left the floor; it does not support “cannot locate itself”.) On blind continuation it sits near chance. A structural code arrives with the where already valid and needs only to rebind. That is the factorisation claim, measured.

The separation is what is doing the work, not the architecture. Two controls make this attributable. The path-integrated and index arms here are parameter-matched to within 0.4% and differ only in whether the phase is driven by the action stream or the index, so the gap is the *driver*, not capacity. And the blind-continuation result survives context destruction — shuffling the explore-phase observations takes it from 0.918 to 0.074 — so the model is using the conjunction it built in context and not a statistical regularity of the task.

Two boundaries, both of which the factorisation predicts. The separation buys nothing when the structural code cannot be *formed*: under rotation-based actions the displacement depends on accumulated heading, a fixed per-token increment cannot represent it, and the measured effect falls from +0.438 to +0.050 (at a 16-epoch recipe with the index arm on the floor) — an index code then does as well, because neither has a usable where. And it buys nothing when there is no structure to share: the effect is a threshold in map extent (-0.010 , $+0.015$, $+0.305$ for 32, 128, 512 occupied cells), so on a small environment, where an episode revisits everything anyway, there is nothing for a structural code to generalise *from*.

What is not shown. These are transfers to a new *instance* of the same structure — a redrawn observation map on the same torus, new motif instances in the same grammar. Transfer across a change of *structure* — a different topology, a different action space, or a graph learned in one environment reused in another — is the claim TEM makes most strongly and is not tested here.

So is *faster* learning: everything above is final accuracy, and no sample-efficiency curve has been measured. Both are the obvious next experiments, and §4 lists what the frame predicts for them.

3 Measurements

The axes of the companion review are separable only where position must be reconstructed from the input rather than read off the index, which is why the measurements below are on navigation and algorithmic tasks rather than on text.

Setup. The primary task is **torus revisit prediction**: an agent takes actions on an $N \times N$ torus, observations are aliased across cells (16 types over N^2 cells, half of them blank), the token stream interleaves actions and observations, and the model predicts the observation at *revisited* cells only. Unless stated otherwise: $N = 64$, one layer, two heads, $d_{\text{model}} = 128$, $n_b = 32$, trained at sequence length 128 for 300 epochs with warmup and cosine decay at $\text{lr} = 10^{-3}$ (the rotation-action and allocentric numbers are the exception: a 16-epoch linear-decay recipe), and evaluated on a held-out observation map. “Accuracy” is next-token accuracy at revisited positions, against a measured always-predict-blank floor of 0.506. Two further tasks appear where noted: **parity** at sequence length L , and **blind continuation**, in which the model explores with observations revealed and then predicts the observation at each cell with observations withheld (chance $1/16 = 0.0625$).

What these measurements assert. Three findings and two negatives, each with the contrast that would have refuted it.

- **F1. The sign of the increment is load-bearing where the task needs a map past the training length, and free.** At matched loss and identical parameter count, at $4\text{--}8\times$ the training length, a signed increment beats an index code by 0.12–0.20 (12/12 seeds); a monotone one beats it nowhere (§3.1). At the training length the monotone arms score 0.90–0.98 against the index code’s 0.80; no arm has been trained at the length where the cost appears. *Refuted by* a monotone arm matching the signed arm on the map task.
- **F2. Rank $r = 2$ fails by search, not capacity.** Trained at $T = 128$, $r = 4$ buys +0.085 at $8\times$ that length for 384 parameters and is flat to $r = 32$. Trained and tested at $T = 1024$ from matched initial weights, training finds the solution within 900 epochs on 0/8 seeds at our shared $r = 2$, 2/8 at a per-head $r = 2$ and 8/8 at a shared $r = 4$, while a rank-2 solution exists and holds (§3.2). What separates them is per-head rank: a block-diagonal $r = 4$ (rank 2 per head) solves 2/8 and a per-head $r = 4$ 8/8 (Fisher and permutation $p = 0.0070$); sharing and W_{out} ’s per-entry scale are unmeasured. *Refuted by* a capacity slope, by the deficit growing with world dimension — which was tested, and it does not — or by the projected rank-2 solution failing to hold under training, which was tested, and it holds.
- **F3. One observable co-varies with F1 and F2, and it reads off the task.** The accumulator’s growth exponent α correlates with cancellation at $r = +0.9995$ across arms (§3.7) — an association, not an explanation — and an unconstrained architecture moves it from 0.591 to 0.967 between the torus and the recency task while every constrained arm stays fixed (§3.8). *Refuted by* the free arm learning the same accumulator on both.
- **N1. The axis on which most mechanisms help is not explained.** Four separate mechanisms help at extrapolated length; α accounts for two and demonstrably not the others,

and the imported long-context account, transposed to a one-layer path-integrated model, is refuted (§3.6). This is the gap, stated as one.

- **N2. An explicit what/where separator works and buys nothing.** Given CoPE’s gate on a signed increment, the model separates action from observation tokens by a factor of four — and gains no accuracy on either task (§3.10). *Refuted by* the gain that F3’s ablation predicted, which is what we expected and did not get.

Three further subsections are supporting rather than load-bearing: what the neighbouring constraints are worth (§3.3), why these axes are invisible on text (§3.4), and a cost/benefit summary (§3.5). Two report on specific mechanisms rather than axes: the published external check (§3.9), which does not discriminate, and where MapPoPE lands (§3.11).

3.1 F1: the sign is load-bearing, and free

Six arms $\times$ 12 seeds, one batch, $r = 4$ throughout, **parameter count identical at 204,757** — the constraint adds nothing and does not touch the rank.

Table 2: The sign ablation, **raw** accuracy. Six arms $\times$ 12 seeds, one batch, identical parameter count (204,757). The construction control **Vanilla_r4**, omitted here for width, is within +0.006 of **signed** at every length — *unmeasured*, which is the condition for the batch to be readable at all. Read this table beside the loss-matched contrasts below: raw and loss-matched point the same way for the signed-vs-monotone comparison and opposite ways for monotone-vs-index.

arm	increment	$T=128$	$T=512$	$T=1024$	final loss
signed	$W_{\text{out}}W_{\text{in}}x$	1.000	0.978	0.922	0.0002
$ \cdot $	$ W_{\text{out}}W_{\text{in}}x $	0.946	0.675	0.558	0.171
softplus	GRAPE-AP form	0.977	0.798	0.584	0.068
CARoPE	$1/(\text{softplus}(\cdot) + 1)$	0.900	0.809	0.645	0.293
RoPE	index	0.799	0.449	0.345	0.784

The raw column is not the contrast of interest, and says so plainly: the monotone arms fit far worse (0.171, 0.068, 0.293 final loss against 0.0002), so a raw comparison against an index arm that fits worse still (0.784) reports convergence rather than representation. **At matched training loss, signed path integration beats an index code by +0.123 and +0.195 at $T = 512$ and $T = 1024$ (12/12 seeds), while monotone path integration beats it nowhere** — -0.028 at $T = 128$ and unmeasured beyond. Note also that the index arm sits at or below the measured always-predict-blank floor of 0.506 at both OOD lengths, so its raw margins are not interpretable as skill. Past the training length, the entire measured value of a content-dependent phase over a plain index clock, on this task, requires the increment to be signed. At $T = 128$ the loss-matched contrast is uninformative — it makes even the signed arm lose to the index code (-0.021 , detectable), because every path-integrated arm fits far better — and raw, every monotone arm beats the index code (0.90–0.98 against 0.80). So the accuracy cost of monotonicity is an extrapolation effect; a monotone arm trained and tested at $T = 1024$ has not been run.

The learned code shows why. The opposition score $\|\Delta(+x) + \Delta(-x)\|/\|\Delta\|$ is 0 when opposite actions cancel exactly and 2 when they are identical:

CARoPE’s published parameterisation reaches 1.98 of a possible 2.00, with its north and east axes all but collapsed onto one another. These models do not merely score worse — they cannot represent a -1 action, and the probe confirms they do not.

Table 3: The learned action code. Opposition is 0 when opposite actions cancel exactly and 2 when they are identical.

arm	oppose _x	oppose _y	$ \cos(N, E) $
signed	0.125	0.106	0.218
$ \cdot $	1.849	1.855	0.587
softplus	1.905	1.885	0.723
CARoPE	1.981	1.977	0.930

At the training length the signed baseline is at 1.000 ± 0.000 , so accuracy there cannot show a deficit of any size. The training-length effect is visible in the *loss* instead: every constrained arm fits strictly worse on 12/12 seeds at identical parameters.

This is a replication in a new regime. The constraint and its consequence for parity are established (the companion review(ii)); what is new is navigation rather than formal languages, and an isolation in which $|\Delta|$ against Δ is a single operation at identical parameter count.

3.2 F2: rank $r = 2$ fails by search, not capacity

The bottleneck r is meant to be the dimensionality of the action space — for a 2D grid, a 2-vector — which makes $r = 2$ sufficient *by construction*. It is not sufficient in practice. Against $r = 2$ at $8 \times$ the training length ($T = 1024$ against a training length of 128; 8 seeds):

Table 4: The rank sweep against $r = 2$, 8 seeds. A step at $r = 2$, flat from $r = 4$ to $r = 32$.

	params added	$T=512$	$T=1024$
$r = 4$	+384	+0.038	+0.085 (8/8, t 3.57)
$r = 8$	+1,152	+0.037	+0.091
$r = 16$	+2,688	+0.028	+0.079
$r = 32$	+5,760	+0.038	+0.095

This is a **step at $r = 2$, not a capacity slope**: flat from $r = 4$ to $r = 32$. It is also an extrapolation number: both arms are at ceiling at the training length, and 94% of the +0.085 comes from short-gap revisits that occur later in the sequence than training reached. Given four dimensions the model still places its actions in a 2-plane (100.0% of spectral energy; 99.96% even at $r = 32$), vindicating the dimensional argument, and the arm means differ in the basis:

Table 5: The learned basis at $r = 2$ and $r = 4$ (arm means, trained at $T = 128$). Descriptive: within $r = 2$ neither column predicts accuracy.

	opposition ($N+S \approx 0$)	$ \cos(N, E) $	obs/action norm
$r = 2$	0.495	0.783	0.139
$r = 4$	0.092	0.175	0.042

On average, at $r = 2$ opposite actions fail to cancel by half the action scale and north and east are nearly parallel. An earlier version read this as the cause (“conditioning, not capacity”); it does not survive. The $r = 2$ codes are bimodal across seeds, and within $r = 2$ neither opposition nor $|\cos|$ predicts accuracy at $T = 1024$ ($r = -0.35$ and $+0.30$).

At matched length it is search. Trained and tested at $T = 1024$ for 900 epochs, every arm built from the same initial weights, 8 seeds: our shared $r = 2$ solves 0/8 (accuracy 0.894), a per-head

$r = 2$ 2/8 (0.885) and a shared $r = 4$ 8/8 (0.998); Fisher $p = 0.0002$ and 0.007 against $r = 4$, so $r = 4$'s advantage is not its initial draws. The per-head arm and $r = 4$ have the same four latent dimensions and identical initial W_{in} , but differ in three ways at once — the rank of each head's content-to-angle map, whether the heads read a shared latent, and W_{out} 's per-entry scale. A second batch, built the same way, separates them with two more arms: a block-diagonal $r = 4$ (the shared $r = 4$ with its cross-head blocks held at zero, so rank 2 per head at $r = 4$'s W_{out} scale) solves 2/8 (0.948), and a per-head $r = 4$ (rank 4 per head, separate latents) 8/8 (0.999). The split is entirely by per-head rank: every arm at rank 2 per head solves 0–2 of 8, every arm at rank 4 solves 8 of 8, and total latent size does not track it. Per-head rank fires (per-head against block-diagonal $r = 4$, Fisher and permutation $p = 0.0070$, Holm 0.028); sharing (per-head against shared $r = 4$, 8/8 against 8/8, permutation $p = 0.59$) and W_{out} 's per-entry scale (block-diagonal $r = 4$ against per-head $r = 2$, 2/8 against 2/8, permutation $p = 0.24$) are unmeasured. The initial-angle-scale worry is answered rather than merely matched: zeroing the cross-head blocks halves the block-diagonal arm's initial increment (std 0.208), but our shared and the per-head $r = 2$ start at normal scale (0.335, 0.328) and fail the same way, so low initial scale is not necessary for failure. The two original $r = 2$ arms are not distinguishable (2/8 against 0/8, Fisher $p = 0.47$): unmeasured. The solution is not out of reach at rank 2: projecting each solved $r = 4$ model onto rank 2 scores 0.9955 frozen, and trained onward from there it stays solved (7/8, against 7/8 for the $r = 4$ control) while from-scratch $r = 2$ reaches 1/8 after 1800 epochs. The representation exists and is held (measured for our shared $r = 2$; for the per-head arm it is inferred); training from scratch does not find it — the same pattern as EM's recency deficit. A search aid partly recovers rank 2 but does not reach rank 4: looping our shared $r = 2$ block $4\times$ at identical parameters and initialisation raises accuracy from 0.894 to 0.973 (permutation $p = 0.0034$) and solves 2/8; four real layers ($4\times$ the parameters) reach 0.990 and 5/8, beating the loop (+0.017, permutation $p = 0.0009$); $r = 4$ is at 0.998 and 8/8. Final losses form three regimes, and nothing at rank 2 enters $r = 4$'s on any seed. The registered verdict is unmeasured, and uncertain rather than negative: the 0.05 solved cutoff falls inside the aids' spread, and at 0.08 the loop would have met its solved-count condition. Since depth beats the matched-parameter loop, this is not search at constant capacity. Scope: two heads, one task, one length, one width, one recipe, budget-scoped (runs still descending count as unsolved). **Which** $r = 2$: our bottleneck is shared across heads and the paper's W_{in} is per head, so at two heads our $r = 2$ has half the paper's latent dimensions. The paper does not state W_{out} 's shape: the per-head arm above is its literal reading, and with a full W_{out} the paper's $r = 2$ at two heads is our shared $r = 4$. The seed standard deviation also falls from 0.064 to 0.012 at $T = 1024$ in the $T = 128$ -trained sweep, which matters because it sets every detectable effect size.

Two further observations. The deficit does *not* grow with the dimensionality of the world: at $D = 5$, $r = 2$ is unimpaired (0.896, its best of three conditions), so the packing account of the companion review does not explain it. And an unconstrained generator cannot be projected below rank ≈ 16 post hoc while $r = 4$ trains fine — trained-at-rank and truncated-to-rank are different objects.

3.3 Supporting: what the neighbouring constraints are worth

Selective RoPE's three additions to MapFormer's generator, measured on the torus at $T = 1024$ and on a parity task at $L = 128$; increments against MapFormer at $r = 2$.

Two readings. First, **the two $\approx 8\text{k}$ -parameter knobs are statistically indistinguishable from each other** (+0.018 and +0.028, both inside their MDEs), so what they buy is most plausibly capacity rather than either named mechanism — and widening the bottleneck MapFormer already has buys the same thing for 384 parameters, $21\times$ cheaper. Second, the convolution is the only knob

Table 6: Selective RoPE’s additions to MapFormer’s generator, against MapFormer at $r = 2$.

constraint removed	params	parity $L=128$	torus $T=1024$
$K = 1 \rightarrow 4$ (causal conv)	+193	−0.020	−0.064 ($t - 1.99$)
$m = r \rightarrow Hn_b$ (full rank)	+7,873	−0.020	+0.058 (7/8)
φ linear $\rightarrow$ gated	+8,193	−0.030	+0.086 (8/8)
all three	+16,385	−0.009	+0.048 ($t 1.36$)
$r = 2 \rightarrow 4$, <i>constraint kept</i>	+384	—	+0.085 (8/8)

negative on both tasks, consistent with the companion review: $K > 1$ costs the homomorphism.

These arms replace the readout $\text{diag}(\omega)W_{\text{out}}$ by τW_2 . For the conv and gate rows the function class is preserved exactly — it is the construction of the companion review — so the difference is initialisation (a geometric frequency ladder against a default draw) plus one scalar; those effects are attributable up to initialisation. The two full-rank rows also add weight normalisation and a bias, so their function class genuinely differs.

A separate probe rules out the natural mechanism for the gate. If the sigmoid worked by suppressing observation tokens — which MapFormer must instead learn to send to $\Delta \approx 0$ — the gate would be near zero on observations. It is 0.416, and the action/observation ratio is $1.35\times$ on the torus where the gate helps and $1.54\times$ on parity where it hurts.

3.4 Supporting: why these axes are invisible on text

Every other paper in the companion review evaluates on language, recall, state tracking, or — for LieRE and Algebraic PE — images, video and trees. Only MapFormer runs grid navigation, and it is the regime in which these axes are distinguishable at all.

The basic separation is large. On a task where the model explores with observations revealed and then continues blind, predicting the observation at each cell, path integration scores 0.730 ± 0.247 ($n = 5, 64^2$) against 0.154 ± 0.018 for an index code, chance 0.0625, with no seed overlap. The axis is path integration and not the encoding: PoPE with path integration scores 0.847 against 0.117 for PoPE with an index (both $n = 3$; only the MapWM pair was extended to $n = 5$, and it fell 0.158 when it was, so read the ratio and not the values). On the standard revisit task under the converged recipe ($n = 8$, held-out map, one batch), the position main effect is +0.243 at the training length (8/8 seeds, MDE 0.038; index RoPE 0.805, path-integrated 0.971) and +0.359 at $8\times$ it. The encoding main effect is *not* negligible: −0.049 at the training length and +0.114 / +0.189 at $T = 512 / 1024$, all detectable. Earlier versions quoted +0.461 and +0.003 from a 16-epoch recipe under which both index arms sat on the floor; that is where the encoding looked inert.

Two boundaries are worth stating. The effect is not a property of navigation as such but of *map extent*: at matched observation aliasing it is −0.010, +0.015 and +0.305 for 32, 128 and 512 occupied cells — a threshold, not a gradient ($n = 3$ per condition; the 512 endpoint that makes it a threshold is the least replicated point, though the 0.290 jump is roughly twice the measured noise floor). And it does not survive rotation-based action spaces: under turn/turn/forward, where displacement depends on accumulated heading, the effect falls from +0.438 to +0.050, which is the companion review’s commutativity limitation made concrete. Recording the absolute displacement instead of the commanded turn restores it to +0.488. All three numbers are from the 16-epoch recipe with the index arm on the floor, and have not been rerun under the converged one.

Table 7: Anchors: what each ingredient costs and buys on the torus task.

mechanism	sets	parameter cost	measured effect
Path integration	phase θ	+448 (+0.23%)	+0.078 parity (16/16); +0.243 torus at $T=128$
Signed increment	sign of Δ	0	largest per parameter; removing it costs $-0.215/-0.280$ at $T=512/1024$
Bottleneck rank	state rank ρ	+384 (+0.19%)	+0.085 at $T=1024$ for $r=2 \rightarrow 4$ trained at 128 (8/8); trained at 1024, solved 8/8 against 0/8
PoPE	magnitude μ	+320	+0.028 <i>with</i> path integration, -0.126 on the index row ($T=128$, converged)

3.5 Supporting: what each ingredient costs and buys

3.6 N1: the length axis, and a refuted import

Transposed to a content-dependent phase the coordinate is the accumulator rather than the index, which gives a prediction: the benefit of the OOD-helping mechanisms should localise to the channels whose period exceeds the trained range of S_t , and ablating those channels should cost *less* at extrapolated length because they are already being read out of distribution.

The precondition holds — at the training length 8–17 of 64 channels never complete a cycle of their own period, and at $8\times$ that length only 1–4 do — so the mechanism could operate. It does not. Ablating the 32 lowest-frequency channels is free at training length (-0.001) and costs 0.14–0.25 at $T = 1024$, on every arm whose baseline is off the floor. **Those channels are more load-bearing at extrapolated length, not less**, which is the opposite of the prediction. Damage is localised in frequency — high-frequency ablation costs 0.42–0.57 — but the plainer reading wins: the low-frequency channels carry position at map scale, and coarse position matters more when the walk covers more of the map.

3.7 F3: one observable co-varies with both

All four arms start from the same accumulator range at training length (88–94) and separate entirely by how fast it grows with sequence length, $\text{range}(S) \sim T^\alpha$:

Table 8: The accumulator’s growth law. All four arms start from the same range at training length and separate by exponent.

arm	$T=128$	$T=1024$	α	opposition score
signed, $r=4$	91.8	269.7	0.518	0.125
signed, $r=4$ (rank sweep)	94.1	279.7	0.524	0.092
signed, $r=2$	88.5	320.5	0.619	0.495
monotone, $r=4$	91.5	649.8	0.943	1.849

A signed, cancelling increment gives $\alpha \approx 0.52$: the accumulator performs a **diffusive random walk**. A monotone one gives $\alpha \approx 0.94$: **ballistic drift**. The last column is the opposition score of §3.1; across the four arms $r(\text{opposition}, \alpha) = +0.9995$.

What α is worth. It is an observable rather than a knob, and it is nearly collinear with the opposition score ($r = +0.9995$), which is read directly off the learned action code. So it carries little information opposition does not, and its contribution is economy: it makes the sign and rank results one finding at two severities rather than two. One use is its own — opposition needs labelled opposite actions and cannot be computed on text at all, while α needs only trajectories, so it is the form this measurement takes in the setting this literature works in. That measurement has not been made.

This is one observable across two separate results, not a mechanism. How completely opposite actions cancel sets whether the accumulator diffuses or drifts; that sets how fast it leaves the range it was trained on; and that co-varies with the rate of degradation with length. The last link is the weak one and is stated as association rather than cause: θ enters through cos, sin, so a larger accumulator is not *prima facie* out of distribution at all, and the standard account of why it would be — untrained low-frequency channels — is refuted here (§3.6). A rank-2 code (opposition 0.495, an arm mean) and a monotone increment (1.849) lie on the same line, but within $r = 2$ opposition does not predict accuracy and the rank deficit is one of search (§3.2), so for rank this is description, not mechanism. The relation is correlational across four arms, and $\alpha \approx 0.5$ for a signed walk is unsurprising in itself; what it rests on is the spread to 0.94 and the agreement with an independently measured quantity.

3.8 F3, continued: the crossover

Every measurement above is on a task that wants a *map*. That makes “a signed increment is better” and “a signed increment suits this task” indistinguishable, and it is the obvious objection to the companion review. The test is a task that counts symbols rather than net displacement. (It does not *require* a clock: a signed rewind of the query token solves it exactly, and a hand-built kernel selects the answer 1423/1423. What it tests is whether giving up cancellation costs anything there.)

The task. A stream of symbols carries interleaved queries; a query names an offset k and the model must retrieve the k -th most recent *content* symbol, where uncounted filler tokens are interleaved at rate 0.5. It is a retrieval, not a decode — asking a MapFormer to *report* elapsed time reproduces a known failure, since the code makes positions comparable and not readable. The filler is what makes k a *contextual* position in CoPE’s sense: the token distance back to the answer is 129.7 ± 10.3 at $k = 64$, so no fixed offset addresses it. Without the filler the task is index retrieval in disguise, and measured that way an index code scores 1.000 while signed and monotone tie. Six arms $\times$ eight seeds, one batch, $k_{\max} = 64$, trained at $T = 1024$; chance 0.0625, shortcut gates at chance, and doubling the budget moves the weakest arm by -0.009 .

Table 9: The crossover. The cost of constraining the increment to be monotone, paired per seed, on the torus and on the recency task. Both are extrapolation readouts, at different ratios: the torus at $8\times$ the training length ($T=1024$, trained at 128), recency at $2\times$ ($T=2048$, trained at 1024, because $T=1024$ is at ceiling).

	torus (a map)	recency (counting)	
monotone – signed, raw	−0.280	−0.004	
monotone – signed, loss-matched	−0.215	+0.026	both inside MDE
seeds	12/12 negative	4/8	

Giving up cancellation costs a fifth to a quarter of accuracy on the map task and **nothing measurable** on the recency task: an interaction of about $+0.28$ — a difference, not an inversion,

and one confounded with the extrapolation ratio until the torus arms are read at $2\times$. The accumulator says the same thing more sharply. The unconstrained arm may adopt either code, and adopts a different one per task, while every arm that *cannot* choose sits at the same exponent on both:

Table 10: The growth exponent is a readout of what the task demanded. Only the unconstrained arm can move, and it does.

arm	α (torus)	α (recency)	Δ
signed, unconstrained	0.591 ± 0.028	0.967 ± 0.009	$+0.376$
$ \Delta $ (monotone)	1.010	0.976	-0.033
softplus (monotone)	1.005	1.005	$+0.000$
CARoPE form (monotone)	1.003	0.992	-0.011

$n = 12$ and 8; the shift has a standard error of 0.009. The constrained rows are the control that attributes it to the *choice* rather than to the two tasks’ statistics.

This is corroboration rather than the finding: the result is the accuracy interaction above, and the α table measures the same thing on the learned code instead of on held-out accuracy. Worth having because the two could have disagreed; not independent evidence.

What supplies the clock, established by intervention. The arm does not make every increment positive — the negative fraction of Δ barely moves, $0.498 \rightarrow 0.478$. It learns a **content gate**: large increments on counted tokens, near-zero on filler, present in 8/8 seeds per head. Gate *strength* does not predict the outcome ($r = -0.34$ across a near-ceiling range), so this was settled by eval-only intervention rather than by more measuring. Removing the filler increments is a no-op ($1.000 \rightarrow 0.9997$); removing the counted ones destroys the task (0.068), which is the positive control. The decisive contrast holds magnitude fixed: replacing Δ with a *constant* on content alone scores 0.783, and with a constant on *every* token — same total drift, differing only in which tokens it lands on — 0.189, a gap of $+0.594$ at 8/8 seeds. A constant increment on content recovers most of the baseline, so the gate is most of the mechanism rather than a component of it.¹

And a fixed index code cannot do it at all. The path-integrated arms reach 0.96–1.00 and the index arms 0.234, a gap of $+0.750$ at 8/8 seeds — larger than the navigation effect of §3.4. The per-offset curve says why: the index arms solve $k = 1$ (0.99) and collapse from $k = 8$ on (0.11–0.33), while every path-integrated arm is flat out to $k = 64$. This reproduces CoPE’s own claim in a different architecture — against contextual position, a relative code’s best is a decaying attention — and is cited here as an anchor rather than as a new result. Note also that a token clock built inside this architecture (0.189, above) lands *below* the index arms, which is what it is.

3.9 The external check, and why it does not discriminate

The crossover is measured on a task we built, so it wants an anchor on a published one. Flip-Flop LM (Liu et al., 2023) is the natural choice — it is the benchmark CoPE uses, and it is our recency task at $k = 1$, with write instructions as the counted class and ignores as the filler. Six arms $\times$ eight seeds, one batch, at that literature’s configuration for the task (4 layers, $d = 256$, 4 heads). Chance is 0.500; evaluation scores the final read only, which is what returns every n-gram order to chance.

¹An earlier condition that simply made the filler count also collapsed, apparently decisively. It is confounded and is not counted: a control that changed *only* the accumulator’s scale, still counting content alone, collapsed just as hard. This model is acutely sensitive to θ ’s magnitude, so any intervention that moves it is destructive for reasons unrelated to counting. The magnitude-matched pair was added afterwards, as a control for that confound.

Table 11: Flip-Flop LM, error % (lower is better). Every contrast is unmeasured, and every arm solves the in-distribution split.

arm	in-dist	OOD dense	OOD sparse
signed, $r=4$	0.00	6.69 ± 3.12	26.06
softplus (monotone)	0.00	7.00 ± 5.83	8.84
$ \Delta $ (monotone)	0.00	8.09 ± 3.20	7.69
RoPE (index)	0.00	8.59 ± 4.54	0.13
plain, index	0.00	10.84 ± 5.41	0.00
CARoPE form	0.03	12.16 ± 5.76	3.12

Path-integrated minus index is -1.23pp against an MDE of 3.99; monotone minus signed is $+2.40$ against 3.97; no pairwise contrast clears its own MDE.

This is not a failure to replicate, and our own data said so before the run. §3.8 reports index accuracy *by offset*: $k = 1$ is 0.99, and the collapse to 0.11–0.33 begins at $k = 8$. **Flip-Flop only ever asks $k = 1$** — the bit of the most recent write — which is precisely the offset at which an index code already works. It varies the *distance* to the governing write, through ignore density, and never the *ordinal depth* of the query; ordinal depth is what separates a content-gated counter from an index. The benchmark is a weaker discriminator for this question than its reputation suggests, and that is why the graded- k task was built rather than inherited.

Two further points rather than one. The comparison is underpowered — seed standard deviations of 3–6pp against differences of 1–5pp — so *unmeasured*, not null. And the OOD-sparse column is a shortcut, not a result: at $p_i = 0.10$ the last bit in the stream *is* the governing write’s bit 95–98% of the time, so “copy the previous bit” nearly solves it. The index arms exploit that and the path-integrated arms do not. It is reported because it was gated in advance, not because it means anything.

3.10 N2: an explicit separator works, and buys nothing

§4.2 argues from the crossover that MapFormer’s separator — a linear bottleneck, in which zero must be reached by cancellation — should be improved by CoPE’s sigmoid, for which zero is the resting state. Built as $\Delta = \sigma(W_g x + b) (W_{\text{out}} W_{\text{in}} x)$, per token and per head, at 258 parameters, initialised open so the model starts as its ungated twin. Four arms $\times$ eight seeds on both the torus and the recency task.

The separator works, verifiably. The learned gate closes to 0.244 ± 0.043 on observation tokens while holding 0.987 ± 0.003 on actions — a ratio of **4.16 $\times$** , with all eight seeds above 3.3. The floor was set in advance from an existing measurement rather than chosen: Selective RoPE’s sigmoid gate reads 1.35 $\times$ on this same contrast and was judged *not* to be suppression. A frozen-gate control reads exactly 1.00 $\times$, which is what licenses reading the rest.

And it improves nothing. Against the ungated baseline: $+0.004$ and $+0.003$ on the torus at $T = 512/1024$, and -0.039 raw / -0.016 loss-matched on recency at $T = 2048$ — unmeasured everywhere, never positive on the task where the crossover’s own intervention showed a gate is worth $+0.594$ when supplied by ablation. It also fits worse (0.041 against 0.012).

The frozen control is what makes this readable. The single detectable contrast in the batch is the gate over its *frozen* twin ($+0.009$, 7/8) — and that is almost exactly the size of the frozen twin’s own deficit against the ungated baseline. What the gate learns is to undo the constant rescale that having a gate imposes. Without that arm, “ $+0.009$ on 7/8 seeds” would have read as a win.

So MapFormer’s bottleneck was already separating well enough, and this corroborates

its design rather than improving on it: a trained ungated model already moves the phase about five times more on actions than on observations, and the explicit gate reproduces that for 258 parameters and a worse fit. The inference behind the experiment does not survive it — that the gate is *load-bearing*, which the crossover’s ablation established, does not imply that the model needs help building one.

3.11 Where MapPoPE lands

This line of work began from a concrete question — MapFormer supplies a content-dependent *phase*, PoPE a content-dependent *magnitude*, and the coordinates are disjoint, so what does imposing both buy? — and the frame exists to answer it in a way that generalises. The answer is mixed, and the shape of the mixture is the useful part.

Table 12: MapPoPE’s record. It is the strongest arm on the task the phase mechanism was designed for, and loses to a pure index code on the task where path integration itself fails.

task	MapPoPE	best alternative	reading
torus revisit, held-out map ($n=8$)	0.994 $\pm$ 0.017	MapEM-os 0.987; MapFormer 0.967	best measured
torus, OOD $l = 2048$ ($n=8$)	0.970 $\pm$ 0.028	MapFormer 0.854	best measured
PoPE’s contribution, loss-matched, 4 grids ($n=8$)	+0.077 to +0.095	—	helps; <i>flat</i> in octaves, so the frequency-count account is refuted
MiniGrid DoorKey-16 $\times$ 16, $T=1024$ ($n=8$)	0.942 $\pm$ 0.017	PoPE + index 0.955 $\pm$ 0.003	loses to an index code
compositional transfer ($n=8$)	0.429 $\pm$ 0.117	MapFormer+hier 0.415 $\pm$ 0.096	tied
enwik8 bpc ($n=3$)	1.3786 (lowest)	PoPE+index 1.3806	numerically best, not established : the margin is under its own checkpoint sd

Three things follow, and the third is the one worth carrying away.

It is the best arm on the task the phase mechanism was built for, at the training length and at 16 $\times$ it. That is a real result and it is where the combination earns its place.

It is not a general improvement. On MiniGrid it is beaten by PoPE with an *index* phase, because rotation-based actions defeat a fixed per-token increment (§3.4) — so the phase half is a liability exactly where path integration itself is. And on text the margin is inside its own noise. A combination inherits its components’ failure modes, not only their strengths.

It fills a cell in the table without supplying what that cell promises. MapPoPE reads as “both slots” in the companion review, and the companion review says a system needing map *and* clock must spend both (true of a scalar accumulator; a rank- r one can cancel in one subspace and count in another). But PoPE’s magnitude is *instantaneous* — read off the current token, not accumulated — so it adds no second accumulator, and measured, it does not move the growth exponent at all ($\alpha = 0.689$ against a baseline 0.710, inside its MDE). MapPoPE is two mechanisms in the phase-and-magnitude slots that supplies *one* answer. The genuinely unoccupied configuration is a signed accumulated phase beside an *accumulated* content-dependent magnitude in softmax attention, and §4 says what to build there.

The rank upgrade does not transfer to it. Every MapPoPE number above is at $r = 2$, because that is the constructor’s default — including the figures that make it the strongest arm on the paper task. Since §3.2 shows $r = 4$ buys +0.085 at $T = 1024$ (trained at 128) on the MapWM family for 384 parameters, the obvious move is to apply it. Measured, in one batch of four arms $\times$ eight seeds:

Table 13: The rank upgrade on MapPoPE, against the same upgrade on MapWM. Both lengths unmeasured, on 5 of 8 seeds.

contrast	raw	loss-matched	verdict
$r=4 - r=2$ on MapPoPE, $T=512$	+0.009	+0.006	unmeasured
$r=4 - r=2$ on MapPoPE, $T=1024$	+0.019	+0.015	unmeasured
<i>the same upgrade on MapWM, $T=1024$</i>	+0.085	—	8/8, $t = 3.57$

A quarter of the effect, and it does not clear its own MDE. There is a mechanism for that in the construction: PoPE sets $n_b = d_{\text{head}} = 64$ against MapWM’s 32, so it already carries twice the frequency channels and the $r = 2$ bottleneck binds correspondingly less. **The consequence runs against what motivated the test: “use $r = 4$ ” is a recommendation for the MapWM family, where it was measured, and not a general one.**

The same batch settles something it was not run for. MapPoPE against plain MapWM at equal rank, *within one batch*, is +0.058 and +0.144 at $T = 512/1024$ on 8/8 seeds — the cleanest confirmation yet that PoPE’s magnitude pays on top of a path-integrated phase, which had previously only been supported across batches. But the ungated MapWM arm is the worst-converging in the set (final loss 0.089, flat on 2/8 seeds against MapPoPE’s 6/8), and loss-matched the contrast falls to +0.024 and +0.082 and stops clearing its MDE. Real in direction; partly a convergence gap, and how much survives it is not resolved at $n = 8$.

4 Predictions

The frame is worth something only if it says what to try next. These follow from the axes rather than from the measurements, and none has been run. Each is stated with what would refute it, so that the section is falsifiable rather than a wish list.

4.1 Mechanisms on tasks they have not been evaluated on

4.2 What to borrow, specifically, for the what/where separation

If the goal is a cleaner separation of the structural code from the sensory one, the table says which mechanisms have a part worth taking and which do not.

Take CoPE’s gate, but not its sign — built, and it did not pay. What follows is what the frame implied; §3.10 reports what happened when it was built. The gate does separate, by a factor of four, and buys no accuracy on either task. The argument is kept because its premise is sound and its failure is informative — the bottleneck was already adequate — but read it as a hypothesis since refuted rather than as advice.

MapFormer’s separator is a linear bottleneck: Δ must reach zero on observations by cancellation in $W_{\text{out}}W_{\text{in}}$, and nothing makes zero a natural output. CoPE’s is a sigmoid, for which zero is the resting state, and its entire purpose is deciding *which tokens count*. The evidence that this is the right borrow is that MapFormer re-derives it unprompted: on contextual counting an unconstrained

Table 14: Untested deployments the frame implies. None of these experiments exists.

mechanism	task	why the frame predicts it	refuted by
PaTH, DeltaNet, RWKV-7	navigation with rotation actions	non-abelian transport is exactly what turn/forward needs, and a fixed per-token increment cannot represent it: MapFormer falls from +0.438 to +0.050 there	landing at MapFormer’s rotate level, which would mean the deficit is not about commutativity
MapFormer	state tracking (A_5 , S_5), formal languages	a signed increment is a parity register — measured +0.316 at $L=16$ — and it keeps the parallel scan, which the delta-rule family forfeits	failing where DeltaNet succeeds, at matched budget
Mamba-3	cognitive-map navigation	the only mechanism holding both slots with a signed rotation; should be the strongest non-attention arm	falling to Mamba-2/GLA’s level, which would mean the rotation is not what navigation uses
LieRE	navigation with rotation actions	its generators need not commute, so it is the one <i>positional</i> code that can express a heading-dependent displacement	needing a commuting special case to train at all
CoPE, CARoPE, GRAPE-AP	navigation	they are monotone, so the companion review predicts they fail on a map task specifically, not generally	matching a signed phase on the torus
MapFormer / MapEM	transfer across a change of structure — new topology, new action space, a graph reused across environments	this is TEM’s strongest claim and the one §2 does <i>not</i> test: everything measured here is a new instance of the same structure	the structural code failing to survive a topology change, which would confine the factorisation claim to re-instantiation
any structural code	sample efficiency , not final accuracy	if a where transfers, a new environment should need fewer episodes, not merely reach a higher ceiling. No learning curve has been measured here	equal sample efficiency with a higher asymptote, which would make the separation a capacity effect rather than a transfer one

increment learns a content gate in 8/8 seeds, and a magnitude-matched intervention shows the gate is most of the mechanism rather than a component of it (§3.8). But CoPE’s gate is $\sigma(\cdot) \in (0, 1)$ and therefore non-negative, which by the companion review builds a clock and not a map. The two mechanisms answer *different* questions — a gate says *whether* a token moves you, a sign says *which way* — and CoPE has only the first. The combination to build is a gate on a *signed* increment. One cost is worth stating: CoPE’s gate is computed per query–key *pair*, which is why it has no θ_t at all and sits outside the frame; a *per-token* gate is the weaker but cheap version that keeps the prefix scan.

Read PoPE as the what-channel, which reframes MapPoPE. §3.11 treats PoPE’s magnitude being *instantaneous* as a deficiency, because it supplies no second accumulator. Under the factorisation question it is the opposite: an instantaneous content-driven magnitude carries sensory information *inside* the positional operator while being structurally unable to corrupt the structural code, and that is measured — adding it does not move the growth exponent (0.689 against 0.710, inside its MDE). So MapPoPE is already a what/where split within the operator: an accumulated signed phase for the where, a non-accumulated content magnitude for the what. It is also the best arm measured on the torus task (0.994 against 0.967). The same property is a defect for a system needing a clock and a virtue for one needing a clean separation, which is the sort of thing only the frame makes visible.

Do not borrow Selective RoPE’s gate for this. It is the obvious candidate — a sigmoid on the phase increment — and it was tested directly. It does not suppress by token class: on the torus it gives 0.560 on actions against 0.415 on observations, a ratio of $1.35\times$ and nowhere near suppression, and on parity it is $1.54\text{--}1.70\times$ in a setting where the gate *hurts*. Whatever it does, it is not separating what from where.

Do not borrow the forget gate for this either. It sits in the magnitude slot and is monotone, so it is a clock (the companion review); it suppresses what is *remembered*, not what updates position. It is the right borrow for a different problem, listed below.

Algebraic PE is the one to take when the structure is known — and it is the most relevant row in the table to a *relational* goal. It maps “the algebraic specification of a domain” to orthogonal operators so that the model “upholds its desired structural properties”, and it covers sequences, grids, **trees, and their compositions**. Two consequences. It expresses structures MapFormer’s abelian $SO(2)$ cannot — a tree is not a product of circles — so it is the natural vehicle for the one prediction in Table 14 that the factorisation thesis most needs and this work does not test, transfer across a change of *structure* rather than of instance. And it is the honest oracle for the separation itself: where MapFormer must *infer* the where from the action stream, Algebraic PE is handed it, so the gap between them prices what inference costs.

TAPE’s idea is already tested here, and it is a null. TAPE contextualises the positional encoding “with sequence content across layers”, refining it at each update — i.e. content-driven *re-estimation* of the where along depth. That is the same object as the refine- θ variant measured in this project, which carried and corrected the position estimate at each pass. It does nothing: $-0.001/ -0.011/ +0.005$ at training length across three noise levels, with no slope in noise, and the learned gate settles near zero, so the optimiser *declines* to refine rather than being unable to. That is a mechanism answer and it applies to this borrow before it is attempted.

HGRN offers a third kind of separation: by timescale, bound to depth. Its forget rate carries a lower bound that “increases monotonically when moving up layers”, so lower layers model short-range and upper layers long-range structure. That is neither a slot nor a gate but a third axis on which what and where could be divided, and it has a concrete form here: MapFormer fixes the same geometric ω ladder in every layer, and binding its range to depth instead is a free change with a published precedent. Untested.

Titans gates the other side. Its memory is written by *surprise* — selection on what is worth storing, not on what updates position. A gate on the where and a gate on the what are complementary, and only the first is discussed above.

And keep the bottleneck. Since W_{in} is the identification mechanism, its rank is the channel the whole distinction passes through, and at rank 2 training rarely finds the torus solution within budget (§3.2); what buys it is per-head rank 4, and a shared $r = 4$, which finds it on 8/8 seeds, costs 384 parameters.

4.3 Combinations worth building

The ordering principle. Every combination above pairs mechanisms that occupy *disjoint* coordinates of the companion review. That is the frame’s one practical prescription: mechanisms in the same slot compete and mechanisms in different slots may compose, and which of the two is happening can be read off the table before anything is trained. MapPoPE is the cautionary case — disjoint coordinates are necessary and not sufficient, because both of its halves answer the same question.

5 Conclusion

Two properties of the content-to-phase map, neither applied as a design constraint in the positional-encoding literature for language and neither tested on navigation before, are measured here. The sign costs nothing and decides whether the accumulator measures net displacement or elapsed path; its accuracy cost appears past the training length. Through a rank-2 bottleneck training rarely finds the torus solution — 0/8, 2/8 and 8/8 seeds for our shared $r = 2$, a per-head $r = 2$ and a shared $r = 4$, trained and tested at $T = 1024$ — although a rank-2 solution exists and is held, so the deficit is one of search; the responsible property is the per-head rank of the map ($p = 0.0070$), with cross-head sharing and W_{out} ’s per-entry scale unmeasured, and looping or deepening the $r = 2$ model partly recovers it without reaching rank 4. Neither dominates the choice of encoding: under the converged recipe the encoding main effect on the torus is $+0.189$ at $T = 1024$, larger than rank’s $+0.085$. Both matter only where position must be reconstructed from the input, which is why they are invisible on text and why the measurements here are not.

Corrected 2026-09-25: this paragraph previously said the two properties “account for more of the variance than the choice of positional encoding”, and that the rank “decides whether the map is well conditioned”. Both rested on superseded numbers (the 16-epoch torus recipe and the $r = 2$ basis reading). *Corrected 2026-09-27:* it also said the responsible property of the bottleneck was not separated; it is per-head rank.

What we could not show is as much of the content, and four negatives are worth stating together because each was a prediction of ours that failed.

The growth exponent that unifies the sign and the rank is nearly collinear with a simpler observable, and its route to length-degradation is refuted rather than established. An explicit content gate separates action from observation tokens by a factor of four and improves accuracy nowhere, which corroborates the design it was meant to improve rather than beating it. The rank upgrade that is worth $+0.085$ at $8\times$ the training length on one family is worth a quarter of that, unmeasured, on the other — so a recommendation we had stated generally holds only where it was measured. And the published external benchmark does not discriminate the mechanism at all, for a reason our own per-offset data gave in advance: it asks only the one offset at which an index code already works.

Table 15: Combinations the frame identifies, ordered by how much evidence already points at them. Only the last has been built (Match-Query, one batch); on the torus the loop has been tested at $r = 2$ only, where it helps accuracy but does not reach $r = 4$ (§3.2).

combination	what it fixes	the evidence pointing at it	refuted by
signed accumulated phase + accumulated content-dependent magnitude (MapFormer + a forget gate), in softmax attention	the empty cell: a map <i>and</i> a clock at once. Mamba-3 holds it in an SSM; nothing holds it in softmax attention	the forget gate’s own accumulator is ballistic ($\alpha = +0.956$) beside the phase’s 0.6, so the clock is already there and unexploited; and it would explain why the gate needs a <i>live</i> λ but not decay (untested: the batch that would test it was deleted before analysis)	no gain on a task needing both, i.e. recency-dependent navigation, which does not yet exist
MapPoPE + a forget gate	its actual deficit: it reads as both slots and supplies one answer (§3.11)	α unchanged by PoPE (0.689 vs 0.710), so the second accumulator is genuinely absent	α still unmoved, which would mean the magnitude slot cannot host a clock at all
explicit content gate + signed phase — as an oracle control , not a proposal	measures what learning the what/where split costs, against a design whose stated purpose is to learn it	on contextual counting an <i>unconstrained</i> signed arm learns the gate by itself, and a constant increment on the gated class recovers 0.783 of 1.000 (§3.8)	the explicit gate not beating the learned one, which would mean the learning is free
signed phase + non-abelian transport	rotation action spaces, without the allocentric recoding that needs known heading	recoding restores +0.488, so the information is sufficient; the question is whether the mechanism can carry it	the parallel-scan loss costing more than the accuracy gained
rank $r=4$ + weight sharing across depth	reliability rather than peak accuracy	measured together in one batch: 0.986 ± 0.020 with 8/8 seeds above 0.941, against 0.833 for the loop alone, 0.421 for $r=4$ alone and 0.416 for neither. The paired interaction (+0.149) is not reliable: same-seed retrains on this task drift 0.185 per seed	— (measured in one batch; listed because it is the template the others are proposed on)

The transfer demonstrated here is to a new *instance* of a structure, not to a new structure, and every number is final accuracy rather than a learning curve. Those are the two things the factorisation claim of §2 actually requires, and neither has been measured.